

NLP Project: 中文口语语义理解

王凯灵 521030910356 卜家梓 521030910395 张致远 521030910377

Abstract

本报告是 CS3602 课程《自然语言处理》大作业项目“中文口语语义理解”的项目实验报告。项目小组成员为王凯灵、卜家梓、张致远。

本文中，我们进行调研，对 SLU 任务本身进行了分析与思考¹。根据处理 SLU 任务的一般思路和对所给样例数据的观察²，我们主要提出了基于数据处理的方法³、基于序列模型的方法⁴、基于预训练模型的方法⁵。

1. 任务分析与讨论

1.1. SLU 任务

口语理解 (Spoken Language Understanding, SLU) 作为语音识别和对话状态跟踪之间的连接模块，将用户输入的文字信息转换成结构化的语义信息，即从对话中提取出关键词。该任务中，输出的语义信息采用对话语义动作形式，即语义动作 (语义槽 1、槽值 1、语义槽 2、槽值 2...)。其中，语义动作包括陈述、询问、肯定和否定等，语义槽则对应对话中的关键词，如导航对话系统中的操作类型、起点名称、终点名称等。

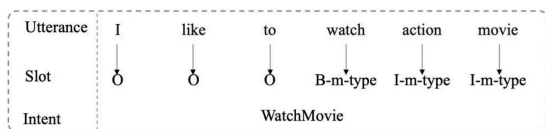


图 1: 口语语义理解

1.2. 常见方法与优化思路

SLU 往往要求我们解析得到三元组。三元组解析可以被理解为一个分割任务和一个分类任务的结合。可以先从输入语句中找出可能为关键词的字段，再将每个字段分类为具体语义。这种思路是合理的，但提示中明确表明使用分类器的方法

效果较差。

如果分割与分类同时进行，则可以将该任务理解为一个序列标注任务。在进行标注时，同时指出了关键字段的类型与位置。为了处理一个输入前后的相关信息，可以使用 RNN 或同类方法处理输入，再通过独立单元预测标签。

如果将输出的标注标签视为一个时序序列，该任务又能理解为一个类翻译任务，即求取从输入序列到输出序列的映射关系。使用常规的 Seq2Seq 方法理应能够解决该问题。

如果使用一些规模较大的预训练模型，也存在绕过序列的概念，通过编解码器处理，直接从编码中提取槽值，即端到端。

SLU 任务的实际应用场景主要包含车载导航、智能助理等。在涉及 SLU 的典型场景中，相比于传统 NLP 特点是单句输入短小，指令性强，往往存在多轮对话，且后续追加往往是对先前信息的矫正和补充。针对这些特点，工程上可能的 SLU 优化方法有：1) 关键词先验匹配：如导航等场景，可能尝试从地图获得词库，对地名进行与标注。2) 历史信息叠加：在同一次对话的后几轮，叠加之前的对话输入。3) 权重调整：即使在同一句话中用户也可能改口，越靠后的字段应获得更高的关注。

2. 数据分析与 Baseline 解读

2.1. 数据分析

本项目提供了一份 SLU 数据集，包含 5119 个训练对话和 895 个测试对话。每份数据含有语音识别结果 (asr_1best, 下称 ASR) 和人工标注结果 (manual_transcript, 下称 MAN)。观察数据发现，数据具有以下特点：

1. 数据噪声强：由于 ASR 之前的系统可能错误捕捉了语音段，以及实际场景中麦克风录音结果质量较差，数据中存在非语音数据、人类无法听辨的语音数据，称之为噪声数据，

如：

```
1 {
2   "utt_id": 4,
3   "manual_transcript": "(unknown)",
4   "asr_ibest": "退出",
5   "semantic": []
6 }
7
```

2. 人工标注与识别结果不一：我们注意到数据中同时包含有语音的识别结果和人工标注的结果，给出的真值语义标签基于人工标注。而模型推理时应该使用自动语音识别得到的文本。由于自动语音识别中产生的谐音错误和混淆音错误等，部分语音语音的自动识别结果和人工标注的结果并不一致。以下称其为“病态案例 (ill cases)”，如：

```
1 {
2   "utt_id": 1,
3   "manual_transcript": "我要去娄桥高速收费站",
4   "asr_ibest": "我要去喽桥高速收费站",
5   "semantic": [
6     [
7       "inform",
8       "终点名称",
9       "娄桥高速收费站"
10    ]
11  ]
12 }
13
```

病态案例会对模型造成误导，尤其是基于序列的方法，所以使用识别的数据进行训练，测试结果应低于使用标注的数据训练。同样，对于测试数据而言，排除病态案例后，基数下降必然导致结果数值上的提高（与模型能力无关）。

表1展示了几种数据使用方式的区别。采用的方式分别为：

1. 使用未经处理的语音识别结果（ASR）数据训练 baseline 中的 LSTM 模型，在测试数据的语音识别部分（ASR）上的测试结果。
2. 使用未经处理的人工标注数据（MAN）训练相同模型，在测试数据 ASR 上的测试结果。

3. 使用去除 (side)(unknown) 等噪音标记的人工标注数据（MAN）训练相同模型，在测试数据 ASR 上的测试结果。
4. 使用 baseline 中训练数据处理方法得到测试数据 ASR 的槽值真值，编码得到的指标。

表 1: *Baselines On LSTM*

method	acc	sp	sr	sf
1	71.40%	81.99%	74.04%	77.81%
2	74.74%	76.00%	78.94%	77.44%
3	75.20%	78.04%	78.94%	78.49%
4	86.40%	99.88%	88.43%	93.86%

其中真值编码得到的指标反映了数据自身自治程度，大致反映直接使用识别结果训练模型的指标上限。

表1中的结果说明人工标注数据 MAN 质量更高，训练模型的效果相较于使用语音识别数据 ASR 训练更好。如果去除 MAN 数据中含有的噪声标记，还能再略微提高模型的表现。

2.2. Baseline Code Review

在 Baseline 给出的样例代码中，使用预先给出的语义槽类型表和 word2vec 表将数据向量化，然后按照一般的序列标注任务思路，使用 RNN、LSTM 或 GRU 处理输入序列后，通过线性层得到标签向量，最后通过反向字典解码得到预测的槽值对。

作为 encoder 的网络结构，对于本任务使用的数据而言，为了增强抗干扰能力，需要结构上相对复杂的网络。同时，由于可以更好地模拟历史信息的权重控制，我们预计 LSTM 模型更加适合该任务。表2检验了这一观点。

表 2: *RNN, GRU and LSTM*

method	acc	sp	sr	sf
RNN	74.86%	78.12%	79.67%	78.88%
GRU	75.15%	77.97%	79.35%	78.66%
LSTM	75.20%	78.04%	78.94%	78.49%

2.3. 实验统一约定

关于2.1-2中提到的数据使用：在真实开发和生产环境中，训练模型时，理应使用更好的训练数据，如加以人工矫正。因此，我们实际**使用了剔除噪声标记（见3.1.1）的人工标注数据 MAN 进行训练**。而测试时，如果只希望测试模型本身性能，同样应使用人工标注数据。但是，我们希望模拟真实环境下 ASR 往往有所偏差的情景，尝试提高模型对抗外部干扰的鲁棒性。因此，我们实际**使用了语音识别数据 ASR 进行测试**。

此外，为确保实验完备，我们也尝试了一些 ASR+ASR 以及 MAN+MAN 的实验组合，详见补充实验部分。

进行实验时，如无特殊说明，均采用 LSTM（如使用 encoder cell），学习率等优化参数和网络参数均采用代码中给出的默认参数进行实验，而且不进行数据增强等数据操作。

3. 基于数据的方法

3.1. 输出重映射算法

3.1.1. 利用先验信息

在1中，我们提到数据中存在较严重的噪声问题和人工标注与识别结果不一的问题。同时，在运行 baseline 模型时，我们发现在很多情况下模型预测出了目标槽值的大部分，而漏掉了位于词开头或结尾的字。同时我们发现 baseline 并没有利用数据集给出的全部信息。如何更加充分地利用数据集的信息，来缓解上述问题，提高模型性能呢？

我们发现 ontology.json 及其关联文件中含有数据集中出现的所有槽值，这定义了任务的边界，是非常有价值的先验信息（下称该信息为“槽值表”）。在模型预测中，由于模型性能问题和数据问题，模型会预测出槽值表中不存在的槽值，但预测值常常与真值接近。如果我们能够依据槽值表，找到和这个预测值最“相似”的槽值并替换，就能够提高模型的性能。这就是输出重映射算法的出发点。

3.1.2. 输出重映射算法

我们利用“**汉字字符串编辑距离**”和“**拼音字符串编辑距离**”的定义，它们将用于刻画两个字符串之间的相似度，来判断槽值表中哪个槽值与预测值最接近。

设两个字符串分别为 s_1 和 s_2 ，其拼音字符串分别为 p_1 , p_2 。则它们之间的汉字字符串编辑距离定义为：

$$dist_{char} = \text{Levenshtein.distance}(s_1, s_2) \quad (1)$$

它们之间的拼音字符串编辑距离定义为：

$$dist_{pinyin} = \text{Levenshtein.distance}(p_1, p_2) \quad (2)$$

式中 `Levenshtein.distance` 代表莱文斯坦距离，其意义是将一个字符串变为另一个字符串需要进行编辑操作最少的次数，是一种常用的编辑距离。莱文斯坦距离的公式表达如下：

$$lev_{a,b}(i, j) = \begin{cases} \max(i, j), & \text{if } (\min(i, j) = 0) \\ \min \begin{cases} lev_{a,b}(i-1, j) + 1, \\ lev_{a,b}(i, j-1) + 1, & \text{otherwise.} \\ lev_{a,b}(i-1, j-1) + 1_{(a_i \neq b_j)} \end{cases} \end{cases} \quad (3)$$

在中文口语语义理解任务中，语料错误识别的原因以字音为主（谐音、混淆音），但由于汉语同音不同字现象大量存在，仅使用字音的相似度可能会出现存在数个编辑距离最小的字符串的现象。因此我们也引入了字形的编辑距离，希望能用识别正确的字形部分作为辅助，以减小同音带来的混淆。我们将上述两种编辑距离加权求和，作为两个字符串间的**综合编辑距离**（ w_1 、 w_2 为 $[0, 1]$ 间的权重， $w_1 + w_2 = 1$ ）：

$$dist = w_1 \times dist_{char} + w_2 \times dist_{pinyin} \quad (4)$$

在语料的解码过程中，我们对比模型预测出的槽值和 `lexicon` 中文件提供的槽值表，如果预测槽值没有落在表中，我们计算预测槽值与表中

每一个槽值的综合编辑距离，并将其替换为综合编辑距离最小（相似度最高）的一个表中槽值。

以上输出重映射算法可以总结为伪代码1。

Algorithm 1 Anti-noise algorithm

- 1: Initialization: corpus set $\{s\} \leftarrow$ raw data
- 2: $s = \text{Noise_Filter}(s)$ (Pre Anti Noise)
- 3: $output = \text{model}(s)$
- 4: Compute $dist(output, slot_value)$ for $slot_value$ in $slot_value_table$
- 5: Find $target_slot_value$ in $slot_value_table$ with smallest $dist(output, slot_value)$
- 6: Update $output \leftarrow target_slot_value$ (Secondary Anti Noise)

Output: $output$

3.2. 结果分析与讨论

将上述输出重映射算法应用到口语理解任务流程中后，再次测试。重映射算法的各项指标与 Baseline 对比如表3所示。

表 3: *ReMap VS Baseline*

method	acc	sp	sr	sf
RM	84.25%	87.25%	87.07%	87.16%
Baseline	75.20%	78.04%	78.94%	78.49%

使用了输出重映射算法后，Baseline 方法的各项指标出现了显著的提升。我们观察到重映射算法的作用主要体现在两个方面：

1. 谐音错误纠正。同音词的拼音字符串编辑距离为 0，算法可以将不在槽值表中的预测值用同音词替换，缓解了错误的自动语音识别结果导致模型无法正确预测的问题。例如，算法将“官员批发市场”替换为“官园批发市场”，“鲁山事件行”替换为“鲁山市建行”，在自动语音识别发生谐音错误的情况下仍然正确预测。
2. 预测错误纠正。人工标注与识别结果一致时，模型也会出现预测错误。如果错误的预测值与真值比较接近，如相较于真值多了一个字或少了一个字，由于两者的汉字字符串编辑

距离很小，算法可以进行替换，纠正模型的部分错误预测。例如将“同同煤集团”补全为“大同同煤集团”，“颍上县谢桥镇政府”补全为“谢颍上县谢桥镇政府”。

值得关注的是，将模型输出强行映射到已有的槽值表中会使得模型只能输出有限的槽值，这显然不利于模型的泛化。但是，对于导航系统等槽值表较为固定且容易获得的口语语义理解任务场景，我们实现的算法能够显著提升性能，总体而言是利大于弊的 tradeoff。

为了解决上述模型泛化能力的损失，我们提出一种解决方案：当预测槽值与槽值表中所有槽值的综合编辑距离 $dist$ 的最小值大于某一阈值 $threshold$ 时，即认定其为新的槽值，不再将其映射到槽值表中的槽值。该方案理论上能够改善输出重映射方法在新槽值上的泛化能力，限于项目篇幅和数据获取，我们尚未实现。

3.3. 历史信息

3.3.1. 多轮对话中的历史信息

许多 SLU 场景（如移动设备、PC 端的语音助手）都涉及人机多轮对话交互。在多轮对话中，人在完成语音输入后，会根据对话系统的回应来补充内容、纠正错误、表达肯定或否定等。对话系统不仅需要处理当前的语音输入，还要和过去的输入结合起来综合分析。由于对话在逻辑上一般是连贯的，利用历史信息在理论上能够提高 SLU 的性能。以本项目的语音导航系统场景为例，如果上一句输入为“我要导航”，则当前输入有更大概率包含目的地等信息，出现“目的地”槽值的概率会明显增加。使用历史信息，能够为模型的预测提供额外的提示，以提高其性能。

3.3.2. 多轮对话记忆网络

我们先重写了原 baseline 中的数据加载模块，随后复现了相关论文 [1] 中的算法以实现历史信息利用。对于每一轮的输入序列，使用两个 RNN 网络 RNN_{mem} 和 RNN_{in} 分别提取其特征。 RNN_{mem} 提取的特征会作为历史特征信息暂

存，而 RNN_{in} 提取当前轮的特征向量。为了判断哪一轮的信息对理解当前轮的语义信息最有帮助，采用了注意力机制，由当前轮的特征向量与每一个历史轮的向量做内积，经过 softmax 获得对应的权重。将其加权求和后再与当前轮的特征向量相加，就得到了综合当前轮和历史轮信息的语义特征。最后通过一个线性层来为当前输入的文本预测标签。

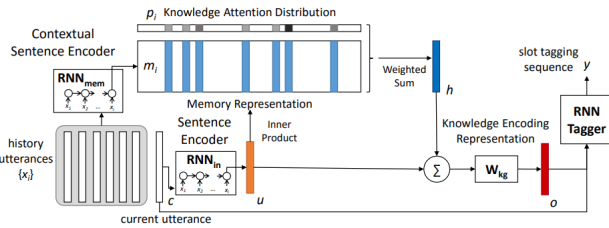


Figure 2: The illustration of the proposed end-to-end memory network model for multi-turn SLU.

图 2: 多轮对话记忆网络

3.4. 结果分析与讨论

表 4: History VS Baseline

method	acc	sp	sr	sf
History	76.20%	78.34%	80.71%	79.51%
Baseline	75.20%	78.04%	78.94%	78.49%

引入历史信息后，模型表现略微提升，说明历史信息能够提高 SLU 的性能。但由于数据集中的多轮对话语料数量较少、前后关联度不高，性能的提升较为有限。

3.5. 数据增强

3.5.1. 语料数据增强

我们注意到，本次项目的语料数据集是较小的，在小数据集上训练容易造成过拟合，不利于模型的鲁棒性和在测试集上的泛化能力。

通过仔细阅读语料，我们发现数据来源应该是导航软件的语音对话数据。在语料数据中，每一种语义槽 slot 对应的若干槽值 value 通常是可以相互替换的。例如语句“请导航到南菜市农副产品批发市场”中语义槽“终点名称”的槽值为

“南菜市农副产品批发市场”，我们将其替换为槽值表中“终点名称”的其他可能槽值如“徐州市火车站”“时代明珠酒店”等后，语句的逻辑依然通顺。同时，我们还注意到“请求类型”“路线偏好”等语义槽也具有类似的特性，即可以通过替换其槽值生成新的合乎人类逻辑的语料。

表5展示了三对原语句和我们通过替换槽值生成的新语句，可见新语句仍然合理。

表 5: 通过替换槽值生成新句

语义槽	原语句	生成语句
终点名称	导航到青岛	导航到浦东机场
请求类型	附近所有的寺庙	周边所有的寺庙
路线偏好	我走国道到东营市	我走高速到东营市

具体的，我们采用的数据增强算法可以总结为伪代码2。

Algorithm 2 Data-Augmentation algorithm

- 1: Initialization: corpus set $\{s\} \leftarrow$ raw data, augment set $\{s'\} \leftarrow \emptyset$
- 2: **for** sentence in $\{s\}$ **do**
- 3: Find all slots that can be replaced in sentence['asr_1best']
- 4: Replace slots with randomly selected values in slot_value_table
- 5: $\{s'\} = \{s'\} + \text{sentence}$
- 6: **end for**

Output: augment set $\{s'\}$

由于语料数据集采集自导航软件的语音对话，用户的导航终点是多样化的，“终点名称”语义槽的可选槽值非常丰富。因此，我们在进行数据增强时，对于“终点名称”语义槽进行了 30 次槽值替换，即每个原语句生成了 30 个更换终点名称的新语句；而对于“请求类型”“路线偏好”等可选槽值相对较少的语义槽，每个原语句仅进行 1 次槽值替换生成 1 个新语句。

3.5.2. 结果分析与讨论

我们将上述数据增强算法应用于数据集的训练语料中，得到增强后的训练集。使用增强后的训练集训练基于 LSTM 的 Baseline 模型，指标得到

了一定的提高，表6展示了使用增强后训练集的实验结果。我们仅展示 Baseline 系列模型的最佳表现，即 LSTM 的水平。

表 6: *Augmentation VS Baseline*

method	acc	sp	sr	sf
AUG	76.65%	79.20%	82.59%	80.86%
Baseline	75.20%	78.04%	78.94%	78.49%

在实际的 SLU 任务中，为了进一步提高模型的性能和泛化能力，应该优先考虑引入新语料以扩充数据集。我们使用数据集增强算法本质上还是在提高原有语料的利用率，但是现实中我们往往无法具备语料结构信息的先验知识，因此更全面更全面的训练语料对 SLU 任务是非常重要的。

4. 基于其他序列模型

4.1. CRF

4.1.1. 条件随机场的作用

按照 baseline 所述方法，输入语句经过 LSTM 和输出层，得到了每个字属于每个 tag 的概率。对于每个字，取概率最大的 tag 最为它的 tag 就得到了最终结果。但我们注意到，tag 之间存在前后文的依赖关系，例如：

- 识别出的语义槽标签必须以 *B* 开头，例如 $B_1 - I_1 - I_1 - I_1 - O$ ，而 $O - I$ 、 $B_1 - I_1 - I_2 - I_1 - O$ 等标注序列是错误的。
- 单个语句中一般只出现一个动词，具体到该任务中为一般至多有一个“操作”槽值。
- 动词一般出现在名词前面，如表示动作的“操作”槽一般出现在表示地名的“终点名称”槽之前。

我们希望模型能够更好地学习到这些依赖关系，参考了相关论文 [4]，引入了条件随机场 (Conditional Random Field, CRF)

4.1.2. 条件随机场原理及实现

在序列标注任务中，CRF 是一个链式的模型。将标签 y_{i-1} 转移到 y_i 的概率定义为标签转移分数 $T[i-1, i]$ 这样就构建了一个标签转移矩阵 T 。发射分数 e_i 取 LSTM 模型的输出结果。定义每个序列的对数得分为

$$score(Y) = \sum_{i=1}^n e_i + \sum_{i=2}^n T[i-1, i] \quad (5)$$

我们希望真值序列的概率在所有序列中最大，即最大化

$$\begin{aligned} \log p(y_{gt}|X) &= \log \frac{e^{score(y_{gt})}}{\sum_{y \in Y_X} e^{score(y)}} \\ &= score(y_{gt}) - \log(\sum_{y \in Y_X} e^{score(y)}) \end{aligned} \quad (6)$$

由于所有可能序列的数量很大，直接计算所有序列的分数的复杂度是指数的，于是采用动态规划的算法，从 0 时刻开始，对每个时刻计算此时到达每个节点的所有路径分数之和，这样计算后一时刻就能够利用前一时刻的计算结果，从而减小计算量。

我们将 CRF 与 LSTM 结合起来，将需要最大化的对数概率 $\log p(y_{gt}|X)$ 取负后作为损失函数，对转移矩阵和 LSTM 参数进行优化，进行模型的训练。而在推理时，先通过 LSTM 得到发射矩阵，再利用训练好的转移矩阵，用维特比算法求出最终的标签序列。

4.1.3. 结果分析与讨论

表 7: *CRF VS Baseline*

method	acc	sp	sr	sf
CRF	76.76%	80.04%	80.71%	80.37%
Baseline	75.20%	78.04%	78.94%	78.49%

引入了 CRF 后，提高了 LSTM 模型的表现。查看训练完成的 CRF 的转移分数矩阵，发现 $O \rightarrow O$ 的分数为 0.49， $O \rightarrow B - inform - poi$ 的分数为 0.069，而 $O \rightarrow I - inform - poi$ 的分数为 -0.59，说明 CRF 通过学习认识到了 $O - I$

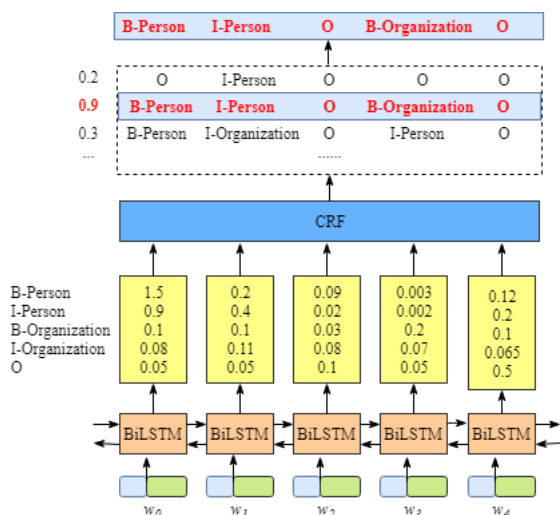


图 3: $LSTM+CRF$ 结构图

标注顺序是错误的，即 CRF 的引入确实能够帮助学习标签上下文之间的约束关系，从而提高模型的表现。

4.2. Transformer

4.2.1. 模型简介

Transformer[6] 是谷歌在 2017 年的论文《Attention Is All You Need》中提出的一种 Seq2Seq 的模型。在 SLU 任务上，我们需要将词编码后的 ASR 结果作为 Transformer 的输入，从 BOS (Beginning of Sentence) 开始逐个生成每个位置上的标签，然后将标签转为所需槽值对。

与 RNN 等传统方法相比，Transformer 中 Self-Attention 的模块提供了关注 token 间关系的能力，从而提取序列中值得关注的部分。而 SLU 所要求提出的槽值，就对应所需关注的区域。Attention 本身通过向量内积的计算提供了反映上下文关系的能力，这使得 Transformer 能够通过关键词和已预测标签捕捉槽的类型，比如输入中出现“经过”二字，则在 encoder 中的注意力模块中，会加强与下文中地点 token 的关系权重，在后续解码中将地点 token 预测为“途经地点”的概率更高，当已经预测得到“操作”的槽时，在 decoder 的注意力模块中产生影响，使得下一时刻的预测结果含有对象和目的地标签的概率更高。

4.2.2. 模型实现

首先，我们需要修改词表，标签表。我们将语句的头 $\langle s \rangle$ 和尾 $\langle /s \rangle$ 分表标注为槽值标签 BOS 和 EOS，并在所有输入语句前后分别添加了头尾。这样，在使用 Transformer 进行与测试时，我们总是可以使用 $\langle s \rangle$ 作为开始令牌。

与一般使用 Transformer 的任务类似，我们采用 TF (Teacher Forcing) 策略作为主要的训练方法。如图 4 所示。该策略不使用模型的完整生成结果进行优化，而是使用真值作为引导，因为我们最终优化目标是产生真值序列，我们只关心前一个预测为真值时模型产生预测的情况，且如果使用模型独立的预测结果，张量将经过模型 token 数的次数，梯度图极为复杂，难以优化。

而在推理时，我们采用使用初始令牌的序列生成，即每次生成一个标签加入结果，不断生成直到满足长度为止。我们使用了规模较小的模型，并且修改了训练数据的权重进行训练，得到最终结果。

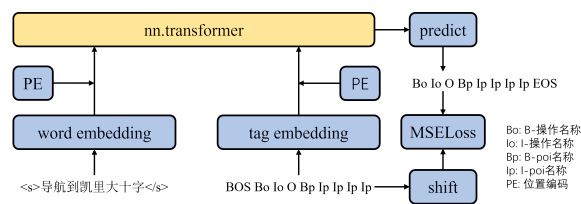


图 4: *Teacher Forcing*

4.2.3. 性能问题与分析

我们首先尝试了不添加开始令牌。此时预测结果的首个标签具有较大随机性，直接导致准确率等于模型完全不进行预测的结果，对应测试数据中没有标签的输入。

起初，我们发现 Teacher Forcing 策略得到的结果较差，具体表现为四项数值中仅有 sp 较高，而 acc 较低（使用默认种子得到）。通过观察具体的预测结果，我们发现由于“操作-导航”在所有数据中出现占比极大，Transformer 很快就能学习其正确标签，故 TP 相当高。而地名等信息，Transformer 没有能力精确学习，往往导致大量的错位、标签不连续等情况。但由于许多地名前紧跟

关键标签，如“去”某地，而一句指令又往往在说完地点时结束，符合此条件的句子可以被正确预测，但模型无法得知地点在何处终止。如果向模型中输入如“我要去 xxx 吃饭”，则模型会将“吃饭”也可能被识别为地名。一般的 Transformer 的确具有该问题。

为了获得改进思路，我们还观察了使用推理的前向过程进行训练的结果。我们意外地观察到这样的训练模式（即 No TF）也能获得较高精度。训练早期与 TF 相差无几，这是由于预测结果靠后的部分梯度复杂难以优化，但前几个 token 尚可进行优化，导致一些以“导航”开头，且不包含其他有效信息的输入可以很快被正确预测，且与 TF 几乎一致。但后续优化过程明显缓慢。

在我们观察标注结果时，我们还意识到数据中存在大量的语音识别错误，但给出了槽值标签的案例。为了了解在该数据上能达到的性能上限，我们计算了槽值标签的确包含在识别结果中的占比，即排除了3.1.1中提到的情况。根据此观察，我们希望降低错误数据对模型的误导。我们在此部分的实验中去除了训练数据中的错误标注（即抹除该部分权重，测试数据保持不变）。为了进一步降低低质量数据造成的影响，我们极大的缩小了模型的规模。由于所给数据数据体量本身不大，此举在所给数据上取得了明显的效果。

4.2.4. 其他改进与结果

transformer 在训练早期经常出现序列不合法的情况，如 I 前面没有 B。我们尝试添加了一个序列规范化，具体方法见算法3。由于没有改变训练本身，实际精度没有发生改变。该方法的意义在于保证模型返回合法序列，避免意外异常。

由于数据中多轮的对话，前文提到导航，后文就会将纯地点标为终点而不是 poi，我们进行了多轮对话的拼接，使用 transformer 一次推理完成的对话。实际上，由于往往在句子中加入了无效信息，对模型造成误导，并没有效果提升。

该方法的结果数据见表8。其中 Concat 为拼接的方法。

Algorithm 3 Formalize algorithm

```

1: Input: Labeled sequence  $s$  by model
2: Output: Formalized sequence  $s_f$ 
3:  $s_f \leftarrow s$ 
4: for index  $i$  : length of  $s$  do
5:   if  $s[i]$  is I-* and  $s[i-1]$  is not I-* then
6:      $s_f[i-1] \leftarrow S^*$ 
7:   end if
8: end for

```

表 8: *OurTransformer VS Baseline*

method	acc	sp	sr	sf
Transformer	75.31%	79.96%	75.29%	77.55%
No BOS	19.11%	0.00%	0.00%	0.00%
No TF	70.06%	72.74%	71.22%	71.97%
Concat	73.18%	78.22%	73.41%	75.74%
Formalize	75.31%	79.96%	75.29%	77.55%

5. 基于预训练模型

5.1. BERT、BERT-wwm 与 RoBERTa

BERT[3] 是 Google 在 2018 年提出的一种预训练的自编码语言模型，它采用了 Transformer 架构的 Encoder 模块，并使用 MaskLM 方法在大规模的文本语料上进行无监督的预训练，学习到了深层的双向（bidirectional）上下文表示，这是一种通用的语言表示，奠定了 BERT 卓越的性能。

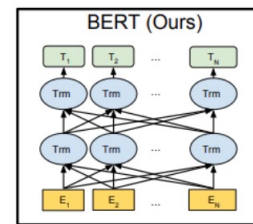


图 5: *BERT* 结构

BERT 在 11 项 NLP 任务中取得 SOTA 结果，已经被广泛地应用到多标签分类、机器翻译、语义理解等领域。

BERT-wwm 是一种 Google 在 2019 年发布的 BERT 升级版，相较于原版的 BERT，BERT-wwm 在原预训练阶段的训练样本生成策略上采用了全词 Mask(Whole Word Masking, wwm)。在此策略下，如果一个完整的词的部分 WordPiece 子

词被 mask，则同属该词的其他部分也会被 mask。

哈工大讯飞联合实验室（HFL）将 wwm 方法（利用 LTP 实现）应用到中文上，使用中文维基百科（简体 + 繁体）作为训练资料，训练得到 BERT-wwm(Chinese)[2] 模型，在阅读理解、情感分析等、句对分类多个中文语言处理任务上表现卓越，促进了中文信息处理的研究发展。

RoBERTa (A Robustly Optimized BERT) [5] 是 BERT 的改进版本，相较于原版的 BERT 主要做出了以下几点改良：

1. 训练数据和策略：使用更多的训练数据，更大的 batch size，更长的训练时间。
2. 损失函数：移除 next predict loss (NSP)。
3. 掩码操作：将静态 mask 操作更换为动态 mask 操作。
4. 编码方式：从 character-level BPE 更换为 GPT2 使用的 byte BPE。

总而言之，RoBERTa 虽然没有完成飞跃式的创新，但是它提出了一套行之有效的 BERT 参数选择和训练策略，能够为基于 BERT 类模型的下游任务的性能提高提供指导。

5.2. 模型对接与词表适配

我们选用基于中文维基百科进行训练的 BERT-wwm(Chinese) 模型作为中文口语语义理解任务的序列标注模型。我们分别使用 BERT-wwm 和 RoBERTa 模型作为中文口语语义理解任务的序列标注模型进行实验。两模型规模相同，编码层均为 12 层，注意力头数均为 12，词表大小为 21128，模型隐层大小为 768。

我们首先将两个模型以相同的方式分别接入已有的训练框架中。BERT 模型输出的是语言的高级隐层表示，隐含了丰富而通用的语义信息。我们自行设计了一个 MLP 网络作为预测头 `output_layer`，用于将通用语义信息转化为序列标注任务所需要的特定表示形式。

在接入 BERT 模型的过程中，我们还进行了词表和分词器的统一。使用预训练模型时，必须

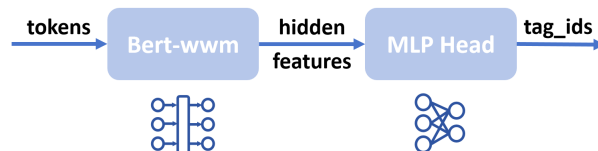


图 6: BERT-wwm + Prediction Head

使用模型训练时所用的词表和分词器，否则输入的语言编码方式与模型训练时不一致，预训练权重不再适配。baseline 使用的词表与我们所用的预训练模型是不同的。不仅如此，两者分词方式也不同。对于中文，它们都是将每个汉字对应一个 token，但对于英文，baseline 将一个字母作为一个 token，而 BERT 模型所用的分词器将一个子词作为一个 token。在处理含有英文的语句时，如果简单套用原有的分词和解码方式，就会造成错位。于是，我们用预训练模型对应的词表替换了 baseline 原有的词表，并修改了解码的部分逻辑，使得预训练模型能够正常地在已有的框架中运行。

5.3. 训练细节

在模型训练时，由于 BERT 模型已经拥有预训练权重，只需用小学习率进行微调即可；而预测头是随机初始化的，需要相对较大的学习率。于是我们为两部分的参数设置了不同的学习率，即：

$$optimizer = \begin{cases} \text{BERT-wwm} & \eta_1 = 2 \times 10^{-5} \\ \text{Prediction Head} & \eta_2 = 1 \times 10^{-3} \end{cases}$$

5.4. 结果分析与讨论

我们对 BERT-wwm 和 RoBERTa 模型的测试结果如表9所示。

表 9: Pretrain VS Baseline

method	acc	sp	sr	sf
BERT	80.78%	83.16%	83.94%	83.55%
RoBERTa	80.67%	83.37%	83.63%	83.50%
Baseline	75.20%	78.04%	78.94%	78.49%

BERT 模型结构合理、参数量大，且经过了

有效的预训练，性能相较于较为简单的 RNN 类模型明显提升。RoBERTa 与 BERT-www 的性能接近，考虑到两模型结构基本相同，仅训练数据和方式有所不同，这样的结果是合理的。

然而在实验中，我们还注意到 BERT 预训练模型所用的词表不含有一些地名中的生僻字（例如“垵”字），这体现出预训练模型虽然泛用但也存在一定的局限性。地名中的不常用字、生僻字相对较多，想要预训练模型在导航系统口语语义理解任务中有更好的表现，可能需要针对这些生僻字配置更大的词表并加入相关的训练数据，重新进行模型预训练。

在预测头的设计上，我们进行了很多尝试，包括单层、多层线性层和引入了 transformer 的预测头，但性能相差很小。推测是由于预训练模型参数规模大而所用数据集规模较小，简单的预测头足以使模型对数据进行有效的拟合，所以增加预测头的复杂度也不再会提高性能。

6. 补充实验部分

6.1. ASR + ASR

当训练数据来源广且基数大时，对所有训练数据进行人工标注将是十分昂贵的，因此有时我们需要考虑训练数据只有语音识别（ASR）结果的情况。

我们使用去除噪声的语音识别结果（ASR）训练模型，并在测试数据 ASR 上进行测试，结果如表10所示。限于报告篇幅，我们仅展示该训练配置下的部分结果。

表 10: ASR + ASR

method	acc	sp	sr	sf
RM+AUG	80.56%	86.26%	83.11%	84.65%
CRF+AUG	79.67%	84.71%	82.57%	83.63%
Baseline	71.40%	81.99%	74.04%	77.81%

6.2. MAN + MAN

我们在实验统一约定中已经提到，如果只希望测试模型性能，也可以使用人工标注数据 MAN

进行测试。我们在此展示训练与测试均使用 MAN 部分的结果（如表11所示）。由于在 MAN 数据上测试指标普遍偏高且在实际任务中使用较少，我们仅展示部分模型的结果。

表 11: MAN + MAN

method	acc	sp	sr	sf
Roberta	92.18%	91.54%	94.79%	93.14%
Baseline	91.40%	92.37%	92.18%	92.28%

7. 项目总结

表12展示了我们在本次项目中所有方法的结果（MAN+ASR 实验组合），相较于 baseline 水平均有不同幅度的提升。其中输出重映射方法的提升幅度是单一方法中最高的，我们将其与 BERT 模型相结合，得到了在本次项目中最高的水平，一并展示于表12中。

表 12: All Our Results

method	acc	sp	sr	sf
RNN	74.86%	78.12%	79.67%	78.88%
GRU	75.15%	77.97%	79.35%	78.66%
LSTM	75.20%	78.04%	78.94%	78.49%
RM	84.25%	87.25%	87.07%	87.16%
History	76.20%	78.34%	80.71%	79.51%
AUG	76.65%	79.20%	82.59%	80.86%
CRF	76.76%	80.04%	80.71%	80.37%
Transformer	75.31%	79.96%	75.29%	77.55%
BERT	80.78%	83.16%	83.94%	83.55%
RoBERTa	80.67%	83.37%	83.63%	83.50%
BERT + RM	85.70%	88.49%	88.95%	88.72%

数据增强方法和对话历史信息利用方法的提升幅度有限，我们认为是训练数据较少且噪声严重导致的。由此可见，在实际的 SLU 任务中，保证训练语料的质量和数量是非常重要的。

CRF、Transformer 等基于其他序列模型的方法也能展现出超过 baseline 的水平，这个结果指出先进的序列模型能够对话料信息有更好的把握。

使用预训练模型能在 SLU 任务上达到相当高的水平，证明预训练的语言模型具有较高的泛用性，其提取的高级语义信息能胜任许多不同任务。

8. 项目分工

总体而言，小组内的三位成员分工是均等的。

8.1. 代码分工

- 王凯灵：Baseline Code Review, Baseline 框架整改, Transformer 实现、Transformer 涉及观察实验代码。
- 卜家梓：Baseline 方法修改, 数据增强算法及代码修改、CRF 实现、RoBERTa 对接、各种参数及数据组合的实验运行。
- 张致远：输出重映射算法及代码、多轮对话记忆网络实现、BERT 对接、各种参数及数据组合的实验运行。

8.2. 报告撰写分工

- 王凯灵：摘要、任务分析与讨论、数据分析与 Baseline 解读、Transformer、审稿与修改。
- 卜家梓：输出重映射、数据增强、基于预训练模型、补充实验、项目总结、审稿与修改。
- 张致远：任务分析与讨论、对话历史信息利用、CRF、基于预训练模型、审稿与修改。

9. References

- [1] Yun-Nung Chen, Dilek Hakkani-Tür, Gökhan Tür, Jianfeng Gao, and Li Deng. End-to-end memory networks with knowledge carryover for multi-turn spoken language understanding. In *Interspeech*, pages 3245–3249, 2016.
- [2] Yiming Cui, Wanxiang Che, Ting Liu, Bing Qin, and Ziqing Yang. Pre-training with whole word masking for chinese bert, 2021.
- [3] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. Bert: Pre-training of deep bidirectional transformers for language understanding. *arXiv preprint arXiv:1810.04805*, 2018.
- [4] Zhiheng Huang, Wei Xu, and Kai Yu. Bidirectional lstm-crf models for sequence tagging. *arXiv preprint arXiv:1508.01991*, 2015.
- [5] Yinhan Liu, Myle Ott, Naman Goyal, Jingfei Du, Mandar Joshi, Danqi Chen, Omer Levy, Mike Lewis, Luke Zettlemoyer, and Veselin Stoyanov. Roberta: A robustly optimized bert pretraining approach. *arXiv preprint arXiv:1907.11692*, 2019.
- [6] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems 30*, pages 5998–6008, 2017.